

CS3002E Compiler Design - Monsoon 2026

Course Project: Details

(Students enrolled in CS4081E Compiler Laboratory will be given a different project.¹)

Objective: Design and implement the front end of a compiler for a simple example language. The compiler will take the source code written in this language as input and produce an Intermediate Representation (IR). The project requires understanding of the theoretical concepts related to lexical analysis, parsing, semantic analysis, and intermediate code generation. The project is to be completed in two stages as given below:

Stage 1: Building a Lexical Analyzer and Parser. Submission Date: 25.09.2026.

Stage 2: Semantic Analysis and Intermediate Code Generation. Submission Date: 15.10.2026.

Students can work in teams of 3-4 members. Evaluations will be conducted after 15.10.26. Assessments may include written tests, implementing small extensions to the work done and viva-voce.

Stage 1: Building the Lexical Analyzer and Parser. Build a Lexical Analyzer and Parser of a compiler for the example language named *Bcs24*. You can use any programming language for implementing the compiler. Use automatic tools to generate the programs for Lexical Analysis and Parsing.

Bcs24 Language Specification: A *Bcs24* program starts with the keyword **BcsMain**, followed by a declaration list and a statement list enclosed within a pair of curly braces. All variables are declared in the beginning of the program. The two primitive data types in the language are **int** and **bool**. The syntax and semantics of statements and expressions are almost similar to that of the *C* programming language.

Syntax:

```
program → BcsMain{ declist stmtlist }
declist → declist decl | decl
decl    → type id ;
type    → int | bool
stmtlist → stmtlist ; stmt | stmt
stmt    → id = aexpr
        | if (expr) {stmtlist} else {stmtlist}
        | while (expr) {stmtlist}
expr    → aexpr relop aexpr | aexpr
aexpr   → aexpr + aexpr | term
term    → term * factor | factor
factor  → id | num
```

Token Specifications:

```
letter → [a-zA-Z]
digit  → [0-9]
id     → letter (letter|digit)*
num    → digit+
relop  → < | > | <= | >= | == | !=
```

Keywords: BcsMain, if, else, while, int, bool

¹Details will be provided by the middle of the semester

A sample *Bcs24* program is given below:

```
BcsMain
{
    int sum; int i; int n;
    n=10;
    i=1; sum=0;
    while(i<n)
        {sum=sum+i; i=i+1};
    sum=sum*10
}
```

Input: Name of the program file to be parsed (as command-line argument)

Output: “Parsing Successful”, if program is syntactically valid, and “Syntax Error”, otherwise.

The parser should detect any kind of syntax error in the program. The exact nature of the error or line number need not be printed. Note that checking if a variable is declared before use is not to be done in this stage.

Submission: On or before **25.09.2026, 10.00 p.m.** Submit the following as a single gzipped file (.gz) through the submission link in the course page:

- Specification files for program generators
- Program files (source code)
- A sample program (not the one given above) for which your compiler prints “Parsing Successful”
- A sample program for which your compiler prints “Syntax Error”

Details regarding file naming conventions will be provided in the submission link description.

Stage 2: Semantic Analysis and Intermediate Code Generation.

Generate 3-address code for a program consisting of one or more assignment statements (for simplicity, flow of control statements are ignored). You should implement the algorithm given in class for 3-address code generation of expressions and assignments. Write the generated code to a text file.

Submission: On or before **15.10.2026, 10.00 p.m.**

Submit the following as a single gzipped file (.gz) through the submission link in the course page:

- Specification files for program generators
- All program files (source code)
- A sample *Bcs24* input program with at least three assignment statements. At least one of the assignments should have a right hand side expression with more than one operators.
- The generated 3-address code as a plain text file.

Details regarding file naming conventions will be provided in the submission link description.

Note on Academic Integrity: Any act of academic dishonesty (including plagiarized/AI generated code), if detected, can result in zero marks for the course project and/or reporting the matter to the department academic council for further action.